Sithu Soe - A17342422

Phyo Thant - A18498144

Title: DevLens

One sentence description: An interactive, live technical interview website featuring an embedded AI coding assistant for candidates alongside a hidden real-time prompting-telemetry and evaluation dashboard for recruiters (Ship it with auth + live URL).

Past project reference: This is a completely new project. The repository can be found at https://github.com/ucsd-cse-genai-programming-sp26/04-student-choice-phyo-sithu-a4

Planned technologies:

Frontend Framework: Next.js, TailwindCSS, Monaco Editor (for the sandbox code interface)

Backend Server: FastAPI (Python), native Python asyncio WebSockets

State & Orchestration: LangGraph / LangChain (if we deem necessary)

Database & Cache: PostgreSQL (for structural data storage), Redis (for WebSocket state, room pub/sub, and rate limiting)

Authentication & Deployment: Supabase Auth, Render

First deliverable:

The first deliverable will be a functional live interview. By the first deliverable, we hope to have a working prototype of a candidate and an interviewer participating in a shared active interview room session where the candidate can use the AI Assistant to help them answer the interview questions. As the candidate progresses through the interview with the AI Assistant, the interviewer and the AI Assistant will evaluate the candidate and their ability to use AI to tackle real world problems. When the interview ends, the admin dashboard should be populated with a summary/metrics of how the candidate performed.

Rough architecture for first deliverable:

Auth & Session Manager (Supabase & Postgres)

- Inputs: User login credentials and role assignment
- Outputs: JWT session tokens and a unique InterviewRoom object containing the room's initial configuration (language, starting code, guardrails, etc...)
- Effects: Secures the application routes and initializes the primary database entries for the session

Browser IDE Component (Next.js & Monaco Editor)

- Inputs: Candidate inputs
- Outputs: WebSocket payloads containing CodeChange events and ChatMessage events.
- Effects: Renders the coding environment on the frontend and pushes real-time telemetry to the server.

Real-Time Sync Gateway (FastAPI & Redis)

- Inputs: Incoming WebSocket payloads from the candidate's IDE.
- Outputs: Broadcasts to specific Redis pub/sub channels.
- Effects: Syncs the live code and chat history to the Interviewer's dashboard with minimal latency, ensuring both parties see the exact same state.

Agentic Chat Engine (LangGraph/LangChain)

- Inputs: The candidate’s chat query, the current Monaco editor code state, and the room's specific guardrail configuration.
- Outputs: An LLM-generated response payload containing text and/or code snippets.
- Effects: Provides the "Copilot" style assistance to the candidate while following the interviewer's guardrails. For instance, it can be instructed to avoid giving solutions.

Hallucination Injector Module

- Inputs: The correct code generated by the agent, and the hallucination probability variable that is set by the interviewer before the interview
- Outputs: A slightly altered version containing logical flaws / syntax errors

- Effects: Forces the candidate to read and debug the AI's output rather than blindly copying it

Telemetry Logger

- Inputs: Timestamps, code differences, chat transcripts, and evaluation flags.
- Outputs: Structured rows appended to PostgreSQL tables (EventLogs, Transcripts).
- Effects: Maintains the entire flow of the interview asynchronously so it can be evaluated later without blocking the real-time WebSocket connection

Scorecard Generator

- Inputs: The complete Transcript and EventLogs fetched from Postgres when the session ends
- Outputs: A structured JSON object grading the candidate across specific metrics.
- Effects: Generates the final post-interview dashboard for the recruiter.

After first deliverable goals:

- Integrate the Piston API or a Dockerized backend to allow the candidate to compile and execute their code against hidden test cases safely.
- Build a component on the Interviewer Dashboard that uses the logged telemetry data to recreate the candidate's keystrokes and AI interactions chronologically.
- Implement a background LLM process that analyzes the candidate's real-time code approach and generates suggested push-back questions on the Interviewer's screen.
- Create a UI where interviewers can set custom variables prior to the session, including adjusting the AI's "hallucination probability," defining token limits, and setting specific guardrails for the role.
- Add a quota system to the chat box UI that restricts how many queries the candidate can send to the AI, preventing them from abusing the tool and forcing them to be strategic with their prompts.
- Cheat-mitigation that flags when candidates copy-paste large blocks from external windows rather than using the built-in tool